

SOFTWARE TESTING · AI-FIRST · REDESIGNED (2026)

Performance & Stress Testing

How fast, how much, how long — and what breaks first. Modern tools, observability, and an AI-first workflow.

10 test types

Metrics & SLOs

Tools 2026

AI-first

IT Faculty, HCMUS · examples in k6 / Gatling / Artillery

Slow is the new down

Users don't wait. A response that is slow enough is indistinguishable from broken.

- › **Speed is a feature** — every extra second of latency costs conversions and trust.
- › **Availability isn't binary** — “up but slow” fails the user just as badly as “down”.
- › **Failures hide until load** — a query fast on 1k rows can collapse on 50M.
- › **Performance is a requirement** — measurable targets ($p_{95} < 2\text{ s}$), not adjectives.

Product vs system performance

The same slowness has two very different root causes — test for both.

PRODUCT — the code

- › Algorithms & data structures.
- › Searches, loops, recomputation.
- › Big-O of hot paths (e.g. sort).
- › Caching & memoization in code.

SYSTEM — the deployment

- › How many servers / replicas.
- › Database schema & indexes.
- › Caching layers (CDN, Redis).
- › Network, I/O, connection pools.

What performance testing measures

Assert on distributions and resources — not a single happy-path number.

Throughput

requests/sec, transactions/sec — how much work in a given time.

Response time

latency as p50 / p95 / p99 percentiles, not the average.

Concurrency

how many simultaneous users before it degrades.

Error rate

% of failed/timed-out requests under load.

Saturation

CPU, memory, I/O, DB connections, thread pool, GC.

Availability

stays reachable and within its latency target.

Six kinds of performance test

Each answers a different question. Pick the ones your risk demands.

Type	Question it answers
Load	Does it meet targets at expected load?
Stress	Where does it break, and how?
Spike	Can it survive a sudden surge?
Soak / Endurance	Does it degrade or leak over hours?
Breakpoint / Capacity	What is the maximum it can handle?
Scalability	Does adding resources add throughput?

Four more you'll be asked about

Beyond the core six — the data, config, fault-isolation and comparison angles.

Volume

Large data, not many users — how do queries/indexes hold as the DB grows to 50M+ rows?

Configuration

Vary the setup (CPU, pool size, cache, JVM flags) — which knob actually moves the number?

Isolation

Replay one known-bad transaction repeatedly to confirm and pin down a specific fault.

Benchmark

Compare against a reference — a prior release, a competitor, or an agreed standard.

One engine, many tests

Same load tool — you change VU count, ramp shape and hold time to get each type.

Type	VU / arrival	Ramp-up	Hold / duration	What you watch
Load	→ expected peak (e.g. 500)	gradual (2m)	hold 5m	p95, error rate
Stress	past peak (3k→6k)	keep climbing	no safe hold	breaking point
Spike	50 → 3000	instant (10s)	short burst	recovery time
Soak	moderate (300)	gradual	hold 2-8 h	leaks / drift
Breakpoint	arrival-rate ↑	continuous	until it fails	max stable load
Scalability	fixed load	same	same, ×replicas	throughput gain
Volume	low VU	—	huge dataset	query vs data size
Config	fixed load	same	same, ×1 knob	which knob moves it

Flip the type by editing the numbers

The k6 stages block below is the whole difference between load, stress and spike.

SAME options.stages — DIFFERENT NUMBERS

```
// LOAD — ramp to expected peak, hold
stages:[{ duration:"2m", target:500 },
        { duration:"5m", target:500 }, // hold
        { duration:"2m", target:0 }]

// STRESS — keep climbing past the peak
stages:[{ duration:"2m", target:1000 },
        { duration:"3m", target:3000 },
        { duration:"3m", target:6000 }]

// SPIKE — instant jump, then drop
stages:[{ duration:"10s", target:3000 },
        { duration:"1m", target:3000 },
        { duration:"10s", target:0 }]
```

DURATION & EXECUTOR DO THE REST

```
// SOAK — moderate load, held for hours
stages:[{ duration:"5m", target:300 },
        { duration:"4h", target:300 },
        { duration:"5m", target:0 }]

// BREAKPOINT — arrival-rate to failure
executor:"ramping-arrival-rate",
startRate:100, timeUnit:"1s",
stages:[{ target:5000, duration:"20m" }]

// VOLUME / CONFIG — hold VU fixed;
// vary the DATA size or ONE infra knob
// (index, pool, cache) per run.
```

eShop scenarios — the six core types

One eShop scenario per type, plus two everyday apps that need the same test.

Type	eShop performance scenario	Everyday-app examples (2)
Load	5,000 shoppers browsing + 500 checkouts at weekday peak; hold, keep p95 < 800 ms.	• Grab at the 6pm dinner rush • Banking app on payday
Stress	Add shoppers past the Black-Friday forecast until checkout errors climb.	• Concert ticket on-sale minute • Course-registration at open
Spike	12.12 flash sale — traffic jumps 20× the instant the sale opens.	• Breaking-news site surge • World-Cup kickoff on a live stream
Soak	Hold moderate load for 48 h over a sale weekend; watch memory & DB pool.	• Zalo running continuously for days • Airline site over a holiday week
Breakpoint	Ramp orders/sec until p95 turns up — the max the cluster sustains.	• MoMo / VNPAY max TPS • Ride-hailing requests/sec per city
Scalability	Double the catalog-service replicas — orders/sec should ~double.	• Video CDN adding edge nodes • SaaS auto-scaling pods under load

eShop scenarios — the four extended types

The data, config, fault-isolation and comparison angles — same pattern.

Type	eShop performance scenario	Everyday-app examples (2)
Volume	Catalog grows to 50M products & 100M orders — is search & reporting still fast?	• Google Photos: years of images • Hospital records over decades
Config	Same load; vary DB pool 50→200 and cache on/off — which setting wins on p95?	• Game server tick-rate / instance size • DB index settings A/B
Isolation	Replay only the slow 'apply coupon' call to confirm and pin the fault.	• Bank: slow interest-batch call • Maps: slow route-recalculation
Benchmark	Compare this release's checkout p95 against last release's baseline.	• Speedometer browser benchmark • Phone app cold-start vs previous model

Load testing — meet targets at expected load

Ramp concurrent users to the expected peak and hold; confirm targets hold.

EXAMPLE — k6

TEST TECHNIQUE

- › **Ramp then hold** — increase VUs in stages to the expected peak.
- › **Correlate** — tie response times to resource metrics.
- › **Find the benchmark** — the point where response times go erratic.
- › **Then tune** — fix the bottleneck; scale vertically/horizontally.

```
// load: ramp to 500 VUs, hold, assert p95
export const options = {
  stages:[
    { duration:"2m", target:500 },
    { duration:"5m", target:500 }, // hold
    { duration:"2m", target:0 } ],
  thresholds:{ http_req_duration:["p(95)<2000"],
                http_req_failed:["rate<0.01"] }
};
```

GOAL Confirm p95 < 2 s and error rate < 1% at the expected peak load.

Stress testing — find the breaking point

Push beyond normal until it fails; learn how it fails and whether it recovers.

EXAMPLE — k6 — beyond capacity

TEST TECHNIQUE

- › **Go past the knee** — increase load until errors climb.
- › **Observe the failure mode** — graceful degradation vs hard crash.
- › **Check recovery** — does it come back when load drops?
- › **Blind spot** — a passing load test says nothing about the edge.

```
// stress: keep climbing past expected peak
export const options = { stages:[
  { duration:"2m", target:1000 },
  { duration:"3m", target:3000 },
  { duration:"3m", target:6000 }, // over-drive
  { duration:"2m", target:0 } ] };

// watch: error rate, timeouts, recovery time
// expect: fail gracefully (429/503), not crash
```

GOAL Know the ceiling and the failure mode — degrade gracefully, then recover.

Two failure modes averages miss

Sudden surges and slow leaks break systems that pass a steady load test.

SPIKE — the flash crowd

- › Jump from low to very high instantly.
- › Simulates a sale, a viral event, a cron burst.
- › Tests autoscaling reaction time.
- › Assert: no dropped requests, quick recovery.

SOAK — the endurance run

- › Moderate load held for hours/days.
- › Catches memory leaks & resource drift.
- › Catches slow DB/connection exhaustion.
- › Assert: metrics flat over time, no creep.

Breakpoint & scalability testing

Find the maximum capacity — then prove that adding resources adds throughput.

EXAMPLE — k6 — steady ramp to saturation

TEST TECHNIQUE

- › **Breakpoint** — ramp until it saturates; record the max stable load.
- › **Scalability** — add replicas; throughput should rise near-linearly.
- › **Watch the knee** — where latency turns up sharply = capacity.
- › **Plan capacity** — size infra from the numbers, not guesses.

```
// ramp continuously; find where p95 turns up
export const options = {
  executor: "ramping-arrival-rate",
  startRate: 100, timeUnit: "1s",
  stages: [
    { target: 2000, duration: "10m" } ],
};
// then: double the replicas, re-run
// throughput should ~double if it scales
```

GOAL Know the capacity number and confirm the system scales horizontally.

Percentiles, not averages

The average hides your worst users. Numbers alone aren't enough — you need trends.

Why the mean lies

- 100 fast + 1 very slow → good average, one furious user.
- p95 = 95% are at least this fast.
- p99 exposes the tail that pages you.

ALSO TRACK THE TREND

- › Latency over time, not one snapshot.
- › Response time vs concurrency curve.
- › Error rate as load climbs.
- › Compare each run to a baseline.

Bottlenecks — what to monitor

A load test that reports only latency can't tell you why. Correlate with resources.

CPU & threads

saturation, context-switching, thread-pool exhaustion.

Memory & GC

leaks, heap growth, long garbage-collection pauses.

Database

slow queries, missing indexes, lock contention, pool limits.

Network & I/O

bandwidth, latency, disk I/O, connection limits.

Queues & caches

queue depth, backpressure, cache hit-rate.

External deps

downstream APIs, rate limits, timeouts.

The performance testing loop

A repeatable cycle — and every step belongs in CI, not a one-off event.



BASELINE Save the first clean run as the reference; every later run is judged against it — regressions become obvious.

Model reality, or you test a fiction

Uniform robots hitting one endpoint tell you nothing about production.

- › **Workload modeling** — mix of journeys in real proportions, not one endpoint.
- › **Think-time & pacing** — real users pause; back-to-back requests distort results.
- › **Realistic data & skew** — hot keys, big accounts, cold cache — not uniform-random.
- › **Production traffic replay** — record real traffic and replay it (PFLB, GoReplay).
- › **Ramp profiles** — open vs closed models; arrival-rate for realistic pressure.

Client-side performance — Core Web Vitals

Server p95 can be great while the page still feels slow. Measure the browser too.

CORE WEB VITALS (Google)

- › **LCP** — Largest Contentful Paint < 2.5 s (loading).
- › **INP** — Interaction to Next Paint < 200 ms (responsiveness).
- › **CLS** — Cumulative Layout Shift < 0.1 (visual stability).

TOOLS

- › Lighthouse / PageSpeed Insights.
- › WebPageTest — filmstrip & waterfall.
- › Chrome DevTools performance panel.
- › Playwright + Lighthouse in CI.

Three measurement lenses

The same request is 'fast' or 'slow' depending on where you stand — use all three.

Protocol-level load

Generate raw HTTP/gRPC/WS traffic — no browser. Cheap, massive VU counts, measures the server/API.

k6 · Gatling · JMeter · Locust

Browser-level load

Drive real (headless) browsers — captures JS, render, and true page timing at lower scale.

Playwright + k6 · LoadNinja

RUM / synthetic

Watch real users in production (RUM) or scripted probes 24/7 (synthetic) — the ground truth.

Datadog RUM · Grafana Faro · Sitespeed.io

SLI · SLO · error budget

Turn 'be fast' into a contract the whole team shares and CI can enforce.

SLI

a measured signal — e.g. p95 latency, success rate.

SLO

the target for that signal — e.g. p95 < 300 ms, 99.9% success.

Error budget

the allowed shortfall — 0.1% may fail; spend it wisely.

The gate

breach the SLO in a test → fail the build / block release.

Load + observability = root cause

A number tells you it's slow; traces & metrics tell you where and why.

- › **Correlate the run** — overlay the load test on system metrics in one dashboard.
- › **Metrics** — Prometheus + Grafana; Grafana k6 ties results to infra metrics.
- › **Traces** — OpenTelemetry distributed traces find the slow span/service.
- › **Profiles** — continuous profiling (Pyroscope) pinpoints hot functions.

Shift-left performance — perf in CI

Catch regressions per-commit with a fast smoke load test and threshold gates.

EXAMPLE — k6 CI thresholds

TEST TECHNIQUE

- › **Small smoke test on PR** — short, low VUs, quick signal.
- › **Thresholds as gates** — fail the build when p95 regresses.
- › **Full stress on schedule** — nightly/release; too slow for every PR.
- › **Trend over builds** — catch slow creep, not just hard breaks.

```
# runs in CI; non-zero exit fails the job
k6 run --vus 20 --duration 1m smoke.js

// smoke.js
export const options = { thresholds:{
  http_req_duration: ["p(95)<300"],
  http_req_failed: ["rate<0.01"] } };
// breach a threshold -> exit 1 -> PR blocked
```

RULE A performance regression should break the build the day it lands — not in production.

From an event to a pipeline stage

Performance testing in 2026 is not a pre-release gate — it runs at every stage.



PERFORMANCE AS CODE scripts, thresholds and SLOs live in the repo and version with the app; results feed the same Grafana dashboards as production, so a test regression and a prod incident look identical.

The load-testing toolbox

Script in the language your team knows; most integrate with CI and Grafana.

k6 (Grafana)

JavaScript · dev-centric · deep Grafana/Prometheus integration

Gatling

Scala/Java/JS · high VUs, efficient · great HTML reports

JMeter

Java, GUI · mature, huge plugin ecosystem · protocol breadth

Locust

Python · code-first, distributed · flexible scenarios

Artillery

JS/YAML · modern protocols: gRPC, WebSocket, Kafka

Cloud / SaaS

BlazeMeter · NeoLoad · LoadRunner · k6 Cloud · global load

The whole toolbox, by category

You need more than a load generator — five layers cover the full picture.

Protocol load (OSS)	k6 · Gatling · JMeter · Locust · Artillery
Frontend / vitals	Lighthouse CI · WebPageTest · Sitespeed.io · DevTools
Cloud / SaaS scale	Grafana Cloud k6 · BlazeMeter · OctoPerf · NeoLoad · LoadRunner
Profiling & APM	Datadog · Dynatrace · New Relic · Pyroscope · Parca
AI-driven	PFLB · LoadForge · LoadNinja · Akamas (autotuning)

Profiling & APM — the 'why'

Load tools say the p95 rose; profilers and APM say which line of code did it.

CONTINUOUS PROFILING

- › **CPU / heap flame graphs** — see the hot function, not just the slow endpoint.
- › **Always-on, low overhead** — Pyroscope / Parca sample production continuously.
- › **Diff two runs** — compare a release against its baseline profile.

APM / OBSERVABILITY

- › **Distributed traces** — follow one request across services to the slow span.
- › **Golden signals** — latency, traffic, errors, saturation on one pane.
- › **Auto anomaly + RCA** — Datadog / Dynatrace flag and rank the cause.

Choosing a load-testing tool

Match the tool to your language, protocols and reporting needs.

If you want...	Reach for
Code-first JS + Grafana dashboards	k6
Very high VUs, low resource use	Gatling
GUI, legacy protocols, plugins	JMeter
Python scenarios, full flexibility	Locust
gRPC / WebSocket / Kafka load	Artillery
Enterprise, cloud, distributed	BlazeMeter / NeoLoad / LoadRunner

AI in performance testing

Let AI draft, watch and explain — you set the targets and judge the results.

Scenario generation

Turn an OpenAPI spec or user story into a k6 script to review.

guide

Anomaly detection

ML baselines flag abnormal latency/error patterns automatically.

sensor

Auto root-cause

Correlate a spike with the metric/trace that moved — ranked hypotheses.

sensor

Why performance testing is hard

The technique is the easy part — these are what actually derail projects.

Environment parity

A test box is not prod — different CPU, data volume and topology skew every number.

Realistic test data

Synthetic uniform data hides hot keys, skew and cache behaviour; masking prod data is costly.

Cost & time

Big load runs burn cloud spend and hours — hard to justify for every commit.

Noise & flakiness

Shared infra, cold caches and network jitter make runs non-repeatable.

Reading results

Correlating latency with the right resource metric needs skill, not just a dashboard.

Third-party limits

You can't stress a payment gateway or SMS provider you don't own — mock or coordinate.

WHAT AFFECTS PERFORMANCE

Every layer of the stack can be the bottleneck

The factors that make an app slow — with concrete, real examples for each.

Performance factors, layer by layer

Slowness hides anywhere from the browser to the disk — test end to end, not one tier.

CLIENT / FRONTEND	render-blocking JS, big images, heavy DOM, no lazy-load
NETWORK	latency & round-trips, no CDN, payload size, no compression
APPLICATION / CODE	algorithms, N+1 queries, blocking I/O, GC pauses, serialization
DATA	missing indexes, locks, connection pool, data volume, no cache
INFRASTRUCTURE / ARCH	scaling model, sync coupling, service fan-out, region, limits

A fast API behind a 5 MB page still feels slow; a fast page on a full-table-scan query still times out.

In the application code

What the program does per request — concrete examples with the tech and the numbers.

Algorithmic complexity

$O(n^2)$ nested-loop dedup on 50k items $\approx$ 2.5B ops; a HashSet makes it $O(n)$.

N+1 queries (ORM)

Hibernate/Django lazy-load: 1 list + 100 row queries = 101 trips $\rightarrow$ eager JOIN.

Blocking I/O on request threads

A sync HTTP call to a slow API holds a Tomcat thread; 200 block $\rightarrow$ server 'hangs'.

Oversized serialization

Returning 5 MB JSON (10k rows) every call; paginate + project only needed fields.

GC / memory pressure

A 32 GB JVM heap $\rightarrow$ multi-second stop-the-world pauses; cut allocations, tune GC.

Repeated work in hot path

Recomputing the same value inside a loop; memoize / cache — find it on a flame graph.

In the database

The data tier is the most common bottleneck — and the easiest to get wrong.

Missing index → full scan

WHERE email=? on 10M rows with no index scans the table (seconds); B-tree index → ms.

Lock contention

Every order updates one counter row → a row-lock queue; use atomic / sharded counters.

Connection pool too small

Pool=10 but 200 concurrent requests → 190 wait; size the pool, add PgBouncer.

Data volume growth

A query fine at 100k rows scans 100M after a year; partition, archive, index.

No caching layer

The same catalog query hits Postgres every request; add Redis / a result cache.

Over-fetching (SELECT *)

Pulling 50 columns to show 3, or the whole table to count; project & aggregate in SQL.

In the system design

How the pieces are wired decides how it scales — or where it jams.

Synchronous coupling

Checkout blocks on the email service; move email to a queue → async, decoupled.

No horizontal scaling

A single stateful app instance; make it stateless → load balancer + autoscale.

Region / topology

App in ap-southeast, DB in us-east → ~200 ms RTT × N queries; co-locate + cache.

Chatty microservices

One page fans out to 30 serial service calls; batch / aggregate / add a BFF.

Single write bottleneck

One Postgres primary for all writes; add read replicas, shard, or CQRS.

Container limits

CPU limit 0.5 vCPU → Kubernetes CFS throttling; right-size requests/limits.

Network, frontend, third-party, tuning

The last mile to the user — and the settings nobody remembers to change.

Latency & round-trips

200 ms RTT × 30 sequential calls = 6 s; batch, parallelize, use HTTP/2 multiplexing.

No CDN / edge

Static assets from one origin across the ocean; serve them from a CDN near users.

Payload size / compression

An uncompressed 3 MB JS bundle; enable gzip/brotli, code-split, tree-shake.

Frontend rendering

5 MB hero image + render-blocking JS → LCP 8 s; lazy-load, compress, defer.

Third-party scripts

A slow analytics/ads SDK blocks the main thread; load async/defer and budget it.

Config & tuning

Default 200 Tomcat threads, no keep-alive, wrong GC; tune pools, timeouts, flags.

REAL INCIDENTS

When performance testing was skipped

Famous outages, the architecture that broke, and the test that would have caught it.

Three outages, three test types

Each maps to a test type from this deck — and to an architecture flaw.

Incident · test type	Architecture root cause	Fix / how to prevent
Healthcare.gov LOAD · capacity 2013 · ~5× expected users	Mandatory account-creation was a single choke point; tightly-coupled tiers; almost no end-to-end load test before launch.	Let users browse before signup (decouple); cache catalog; scale tiers independently; run realistic load tests pre-launch.
Ticketmaster · Taylor Swift SPIKE 2022 · 3.5B requests, ~4× peak + bots	A flash crowd plus bots hit checkout with no waiting room or admission control; the buy path saturated.	Virtual waiting room / queue-based leveling; bot filtering; load shedding (429) so the core stays up.
Slack SPIKE + AUTOSCALE 2021-01-04 · first Monday back	Cold-cache thundering herd; an AWS Transit Gateway did not autoscale fast enough; the provision-service then hit open-files & quota limits so scale-up stalled.	Pre-scale the gateway before peaks; load-test the provisioning path; fix health-checks & OS/quota limits; add headroom.

Three more — breakpoint, soak, scale

Capacity limits, slow leaks, and scaling that doesn't scale.

Incident · test type	Architecture root cause	Fix / how to prevent
AWS Kinesis (us-east-1) BREAKPOINT · capacity 2020-11 · 17-hour outage	Adding servers grew an n^2 mesh of connections; each new server pushed every server past the OS thread limit — a hard ceiling nobody had measured.	Measure the breakpoint before prod; raise limits; cut fan-out; cellular / sharded fleet so capacity grows linearly.
Memory-leak degradation SOAK · endurance common · OOM every few hours/days	A heap or connection leak grows under sustained load; latency drifts up, then the service OOM-restarts — invisible to a short load test.	Run a soak test; watch memory/GC/connections for upward drift; fix the leak; cap heap; recycle workers; alert on the trend.
Pokémon GO launch SCALABILITY 2016 · ~50× forecast traffic	Demand far exceeded the capacity model; a write-hot data path became the bottleneck that adding app servers could not relieve.	Model capacity from real forecasts; shard / use horizontally-scalable stores; managed autoscaling; verify throughput rises with resources.

Architecture bottleneck → remediation

The recurring root causes behind the incidents — and the standard fix for each.

Bottleneck	Architecture cause	Standard fix
Thread-pool exhaustion	unbounded fan-out, blocking calls	bulkheads · async I/O · cap threads
Connection-pool exhaustion	pool too small; slow queries hold conns	size pool · timeouts · circuit breaker
DB slow under load	N+1 queries, missing index, locks	indexes · batch · cache · read replicas
Thundering herd	cold caches; synchronized retries	request coalescing · jitter · warm cache
Retry storm / cascade	aggressive retries amplify load	backoff+jitter · circuit breaker · budgets
No load shedding	accept everything until collapse	queue leveling · admission control · 429
Autoscale too slow	reactive scaling lags the spike	pre-warm · predictive scale · headroom
Single hot resource	one DB/leader/region bottleneck	shard · replicate · partition · CDN

How one cold Monday cascaded

A textbook chain: a spike meets an architecture that can't scale in time.



THE FIXES

- › **Pre-scale ahead of peaks** — request gateway capacity before predictable surges (holidays, launches).
- › **Load-test the control plane** — the provisioning/autoscaling path itself must be tested, not just the app.
- › **Remove hidden dependencies** — monitoring shouldn't die with the data plane; keep dashboards on a separate path.

Resilience patterns = your fixes

Each pattern answers a specific failure the tests above expose.

Cache / CDN

offload reads; kill repeated work → fixes DB & catalog overload.

Autoscale + pre-warm

add capacity ahead of the spike → fixes slow reactive scaling.

Load shedding

reject early (429) to protect the core → fixes total collapse.

Circuit breaker

stop calling a failing dep → fixes cascades & retry storms.

Bulkhead / pool limits

isolate resources → fixes thread/connection exhaustion.

Queue-based leveling

buffer bursts, process steadily → fixes spikes & herds.

Pitfalls that make a perf test lie

A green performance report can be as misleading as a green flaky test.

- › **Averages, not percentiles** — hides the slow tail that actually pages you.
- › **No warm-up / cold cache** — first-run numbers aren't steady-state numbers.
- › **Unrealistic load model** — one endpoint, no think-time, uniform data.
- › **Ignoring the frontend** — fast API, slow page; users feel the page.
- › **One-off, not in CI** — a number with no baseline and no trend proves little.

RULE Test the distribution under a realistic model, on a baseline, in CI — or you're measuring a fiction.

A free stack + an AI-first workflow

Everything below is free / open-source and runs on a laptop and in CI.

FREE STACK

- › k6 — scripting & thresholds
- › Grafana + Prometheus — dashboards
- › OpenTelemetry — traces
- › Lighthouse — frontend vitals
- › GitHub Actions — run in CI

AI-FIRST WORKFLOW

- › AI drafts the k6 scenario from the spec.
- › You review the model & data.
- › Run in CI with SLO thresholds.
- › AI flags anomalies & ranks causes.
- › You decide the fix — then re-baseline.

TAKEAWAY

Performance is a distribution, in CI

- 10 test types: load, stress, spike, soak, breakpoint, scalability, volume, config, isolation, benchmark.
- Assert on p95/p99 and resources, not the mean.
- Model reality; baseline; run in CI.
- Turn 'be fast' into an SLO the build enforces.
- **AI drafts & explains; you set targets & judge.**

READING

k6 · Gatling docs

Google Core Web Vitals

Google SRE book (SLOs)

Grafana k6 + OpenTelemetry

Pyroscope · Datadog APM

Deliverable idea: a k6 suite for the EShop API — load + stress + a CI threshold gate on p95.

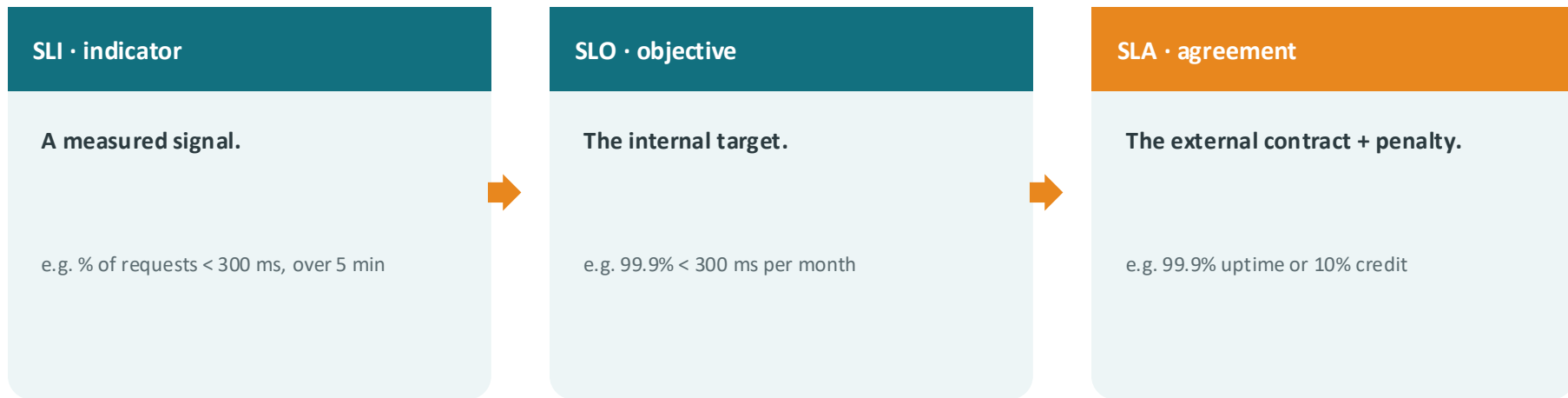
APPENDIX

Concepts & abbreviations

SLI·SLO·SLA, p95/p99, APM, RUM, HAR, SRE, OTel — every term in this deck, defined once.

SLI → SLO → SLA + error budget

From a measured signal, to an internal target, to a contract with penalties.



ERROR BUDGET = 100% – SLO. 99.9% $\Rightarrow$ 0.1% $\Rightarrow$ ~43 min/month of allowed failure; when it's spent, freeze risky releases.

Latency & throughput metrics

How fast and how much — the numbers you assert on.

Term / abbrev.	What it means
p50 / median	Half of requests are faster than this — the 'typical' user.
p95	95% of requests are at least this fast; the slow-but-common tail.
p99	99%; the worst 1% — the tail that pages you. Assert on this, not the average.
TTFB	Time To First Byte — server responsiveness before content arrives.
TTFT	Time To First Token — for streaming/LLM: wait until the first token.
E2EL	End-to-End Latency — total time for the complete response.
Throughput (RPS/TPS)	Requests / transactions per second the system sustains.
Goodput	Throughput that MEETS the SLO — the honest capacity number.

Reliability & SRE

The vocabulary of running services to a target.

Term / abbrev.	What it means
SRE	Site Reliability Engineering — running ops with software; owns SLOs.
SLI	Service Level Indicator — a measured signal (latency, success rate).
SLO	Service Level Objective — the internal target for an SLI.
SLA	Service Level Agreement — an external contract, with penalties.
Error budget	Allowed failure = $100\% - \text{SLO}$; spend it, then freeze risky releases.
Availability 'nines'	$99.9\% \approx 8.8 \text{ h/yr down}$ · $99.99\% \approx 52 \text{ min/yr}$ · $99.999\% \approx 5 \text{ min/yr}$.
MTTR	Mean Time To Recovery — how fast you recover from an incident.
MTBF	Mean Time Between Failures — how often it breaks.

Observability & tooling

How you see what the system is doing.

Term / abbrev.	What it means
Observability	Infer internal state from outputs — the 3 pillars: metrics, logs, traces.
APM	Application Performance Monitoring — Datadog · Dynatrace · New Relic.
RUM	Real User Monitoring — performance measured in real users' browsers.
Synthetic monitoring	Scripted probes run 24/7 from fixed locations — the canary.
Distributed tracing	Follow one request across services to the slow span.
OTel	OpenTelemetry — vendor-neutral standard for traces/metrics/logs.
HAR	HTTP Archive — a JSON capture of all browser network requests (waterfall).
Profiling / flame graph	Sample the running code to find the hot function.

Web, load-testing & architecture

Frontend vitals, load-test terms, and common bottleneck names.

Term / abbrev.	What it means
LCP	Largest Contentful Paint — main content loaded (< 2.5 s = good).
INP	Interaction to Next Paint — responsiveness (< 200 ms = good).
CLS	Cumulative Layout Shift — visual stability (< 0.1 = good).
CDN	Content Delivery Network — caches assets at the edge, near users.
VU	Virtual User — one simulated concurrent user in a load test.
Arrival rate	Requests/sec started (open model) — pressure regardless of speed.
Think-time	The pause a real user takes between actions.
N+1	1 query + N per-row queries — a classic ORM performance bug.